Sonder

A trip-planning system built by people tired of trip-planning systems

---

1. Abstract

The idea for Sonder came from a frustration our lead has carried for fifteen years of planning his own trips. Every existing platform optimises for the wrong thing. TripAdvisor surfaces what coach tours stop at; the top three things to do in any city become the same three things on every list. Google Travel knows the budget but not the mood. Instagram knows the mood but not whether a saved spot is closed on Tuesdays. Airbnb solves where you sleep and stops there. None of them — not one — solves the harder problem of *who you travel with*. Solo travellers get nothing. Couples get hotel suggestions. Anyone who has ever wanted to share a trip with a stranger they actually click with falls back to hostel bars and dating apps repurposed badly.

We built Sonder to be the platform that didn't exist. It does three things at once that no incumbent does together: it generates a day-by-day itinerary that names *real* places — the café with the rosé-coloured awnings on Largo do Carmo, not "explore the old town" — grounded in a hand-curated venue corpus; it matches solo and couple travellers against a measurable compatibility surface rather than swiping over bios; and it sustains a believable social layer through the cold-start phase by seeding a population of LLM-designed synthetic travellers who post, open trips, and message real users about their plans.

The engine underneath is a multi-provider language-model stack. Anthropic Claude runs primary, OpenAI fallback, split across two tiers: Haiku for the fast persona voice, Sonnet for the deep generation work, plus a dedicated validator tier on either side. Image generation handles synthetic-persona portraits. Voice synthesis powers in-chat audio. A 1536-dimensional embedding space holds everything from city contexts to traveller personalities. The whole pipeline streams day-by-day, so a user sees day one of an itinerary within fifteen seconds while later days are still being written, and a validator engine catches assistant-voice leakage, hallucinated venues, and broken persona consistency before any of it lands on screen.

The numbers we care about: itineraries with effectively zero negative-constraint violations against ten-to-twenty-five percent on a prompt-only baseline; persona-grounded chat replies in under two seconds at one-twenty-fourth the cost of a large-model approach; a co-traveller match score calibrated so that a value of 0.6 actually means roughly a sixty percent chance the other side accepts — rather than the uniformly high cosine numbers that make compatibility scoring meaningless on every other matching product we have used.

This report walks through the data the system runs on, the models that drive it, the architecture that deploys it, and the integration battles the team fought along the way.

---

2. The data underneath everything

2.1 Four planes of data

Sonder is not a model trained on a static dataset. It is a live system that produces and consumes data across four planes simultaneously, and the architecture decisions that mattered most came from understanding where each plane lived and what it was good for.

- **The reference corpus** — destinations and activities curated by hand, embedded once, queried thousands of times. Roughly fifty cities and around five hundred activities. We hand-curated rather than scraped because every existing scraped travel corpus reads like a search-engine results page.
- **The synthetic traveller population** — 192 solo travellers and 18 couples, each one designed by a language model under tight diversity constraints. Each carries a written backstory, a voice anchor, two or three quirks, an inferred emotional signature, and a stylised portrait. They live in the same vector index as the destinations.
- **The operational data plane** — what real users produce as they use the product: profiles, generated itineraries, chat sessions, journal entries, social posts, shared-itinerary negotiation history.
- **Telemetry** — every language-model call instrumented, every validator outcome logged, every match score recorded with its full feature breakdown. This is the substrate for phase-two ranker learning, and it is the layer the team trusts to catch LLM regressions before users do.

2.2 The frameworks we built on

The team did not invent the psychology of travel. Three pieces of prior work shaped Sonder's foundations and deserve naming explicitly.

Push-Pull Motivation Theory. Introduced into travel research by *Dann (1977)* and extended by *Crompton (1979)*, the theory frames travel as the product of two distinct motivational forces. **Push factors** are intrinsic and traveller-side — why a person leaves home. **Pull factors** are extrinsic and destination-side — what attributes of a place draw them specifically. We implemented this as a twelve-dimension persona space: six push dimensions, six pull dimensions, each defined by a substring-matched keyword set rather than a single trigger word. The phrase *"out of comfort zone"* counts as one signal toward *adventure_novelty*, not three accidental hits across unrelated dimensions. The whole point of the keyword-list-not-single-word approach is auditability: when the system tells a user they score high on *escape_reset*, the team can point at the exact phrases in their input that produced the score. No black-box psychology assignments.

Push (why they travel)	Pull (what they want from the place)
escape_reset — disconnect, recharge, leave routine	*nature_outdoors* — landscapes, weather, physical settings
adventure_novelty — push themselves, first-time experiences	*culture_history* — heritage, museums, local arts
connection — share with the people they love	*food_drink* — local cuisine, regional specificity

Push (why they travel)	Pull (what they want from the place)
reflection — process, gain perspective	*nightlife_social* — bars, clubs, live music
curiosity — go deeper than the guidebook	*comfort_luxury* — refined service, high-end stays
prestige_reward — milestone trips, dream destinations	*exploration_local* — neighbourhoods, daily-life immersion

GoEmotions. *Demszky et al. (2020), arXiv:2005.00547* — a dataset of 58,000 Reddit comments labelled across 27 fine-grained emotion categories. Sonder does not ship the 58,000 labelled rows. What the system uses is the label vocabulary itself. Each of the 27 emotions gets a one-line tone-anchored gloss — *realization* is *"a quiet click — coming to understand something"*, not a dictionary definition — and the team embeds those 27 glosses once at process start using the same 1536-dimensional embedding model that powers the rest of the system. A user's free-text input is then classified by cosine similarity against those 27 anchor vectors. The result is defensible emotion scores, no separate classifier to retrain, no model registry to maintain, and everything lives in the same vector space as every other signal in the product. The tone-anchored gloss choice is deliberate: dictionary-style definitions cluster too tightly in the embedding space, while tone anchors crisp it.

An eight-key emotional-signature taxonomy. Synthesised by the team specifically for travel personas, this is the coarsest of the three psychology layers — eight archetypes (*story_collector, reset_seeker, aesthetic_pilgrim, depth_diver, energy_chaser, ritual_keeper, quiet_observer, threshold_walker*). The inferrer picks one key per user based on the GoEmotions distribution over their free text plus their structured persona answers, attaches a confidence level (low / medium / high), and the signature becomes private framing for every persona-voiced surface. The key itself is never shown to the user — only the derived emotional-tone phrase, such as *"soft afternoon energy"*, surfaces in the UI. The taxonomy went through four iterations before the team locked it. Earlier versions had too much category overlap; the final eight produce meaningfully distinct chat-reply registers against the same synthetic persona.

The full personality stack is three lenses at three granularities: 27-label fine emotion, then 12-dimension push-pull motivation, then 8-key voice signature. Each consumer in the system chooses the lens appropriate to its task. The chat-reply prompt reads the signature; the matcher reads the PPM space; the persona-reveal copy reads all three.

2.3 The travel APIs we integrated and the failure modes each one had

A travel product without real data about real places is fiction. The team integrated five external APIs into Sonder, and every single integration revealed a failure mode that required engineering around it.

Wikipedia REST API and Wikimedia Commons power destination overview paragraphs and primary imagery. Wikipedia's article-summary endpoint returns the lede paragraph plus the article's infobox image, which is sourced from Wikimedia Commons under permissive licensing. The integration looked trivial until it was tested on a region rather than a city. Wikipedia returns whatever the article's infobox image is, and for *"Patagonia"* that image is a map of southern South America with the region shaded gold — a Wikimedia Commons SVG rather than a photograph. The cinematic reveal screen was ready to ship, and every region-shaped query was rendering a Wikimedia location map as the hero photo. The fix took an evening: a

URL-substring rejection filter that drops any `.svg` (almost always maps, flags, or coats of arms from Wikimedia Commons) plus any URL containing *map*, *karte*, *location*, *locator*, *satellite*, *topographic*, *flag_of*, *coat_of_arms*, or *seal_of*. The fix shipped alongside a 14-day client-side cache, with the cache key bumped so users with already-cached maps would refetch fresh on next load.

Pixabay is the fallback when Wikipedia/Wikimedia returns nothing usable, and the source for the cinematic destination-reveal montage. Pixabay's photography library is contributor-uploaded and licensed under the Pixabay Content License, which permits commercial use without attribution. The integration required server-side proxying because exposing the key client-side would burn rate limits to abuse. It required a country-less retry path because *"Patagonia Argentina"* sometimes returns zero hits while *"Patagonia"* returns thirty. And it required an in-process cache keyed by query and count so that a refresh of the reveal screen does not re-pay quota for the same five photos. The cache is what makes the cinematic montage — five photos cycling every 1.2 seconds with a slow zoom — feel free on every subsequent view.

The two image sources are deliberately layered. Wikimedia gives us editorial-quality, well-attributed imagery when available; Pixabay backfills with destination-shaped travel photography for regions, neighbourhoods, and any place Wikimedia's curated infobox does not cover. Neither source needs an attribution string baked into the UI under their respective licenses, which keeps the interface clean.

OpenWeather provides current conditions by latitude and longitude. Cheap, fast, well-behaved.

Nominatim (OpenStreetMap) handles geocoding from city/country tuples to coordinates. OSM's terms of service request no more than one request per second from a single source. We wanted to respect that rather than risk being rate-limited mid-traffic-spike. The integration wraps Nominatim in a process-wide token-bucket rate limiter and a 30-day disk cache. A repeat user planning a known destination hits the network for essentially nothing on subsequent loads.

ExchangeRate API handles currency conversion at the input boundary. All internal cost fields in Sonder are USD; conversion happens once when the user submits a budget in their preferred currency. The integration has a 3-second timeout and a hardcoded fallback table for thirty currencies, so the product does not break when ExchangeRate's free tier flakes overnight.

A consistent pattern emerged across all five: **cache aggressively, fail soft, never let a partner outage break the product.** Wikipedia is down? Use Pixabay. Pixabay is down? Fall back to a gradient hero. OpenWeather slow? Skip the weather line. Nominatim rate-limited? Cache hit. ExchangeRate offline? Use the hardcoded table. A user never sees an error screen because a third-party API decided to have a bad afternoon. For a product where users are mid-flow planning the trip of their year, this guarantee is non-negotiable.

2.4 The synthetic-traveller corpus — why we built a population from scratch

The matching surface needed a candidate pool, and real users would not exist on day one. The two honest options were to launch with an empty matching screen and pray for organic adoption to fill it, or to seed a population. We chose the second, but the way the team built it is what matters.

The diversity matrix is locked: sixteen cities across five continents (New York, Mexico City, Buenos Aires, Bogotá, London, Paris, Berlin, Lisbon, Istanbul, Lagos, Cape Town, Dubai, Mumbai, Bangkok, Tokyo, Seoul), three age buckets (20-30, 30-40, 40-50), two genders in a 50/50 hard-locked split, two personas per cell. The total is 192 solo travellers. A separate matrix produces 18 male-female couples for couple-mode matching. The cell-density choice — two per cell, not one — was made after a real production incident detailed later in section 3.

Each persona is generated through a **two-stage blind-writer pipeline**. The first language model receives only what a novelist would receive: the city, the age bucket, the gender, and four persona-question option keys to pick from. It writes the character — name, voice anchor, small-thing answer, quirks, appearance descriptor. **It never sees the PPM dimensions, the emotional-signature taxonomy, or any of the matching feature names.** The second stage, the inferrer, then runs the written persona through exactly the same pipeline a real user would go through, assigning PPM dimension labels and an emotional signature blind to what the writer was *supposed* to produce.

This is the most consequential design decision in the system, and it is the reason Sonder's personas do not read as obviously machine-generated. Information starvation as quality control. A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest. Portraits are generated from the appearance descriptor only — painterly, explicitly not photorealistic, which biases the population the right way for the *"Sonder Curated"* disclosure pattern. End-to-end cost is roughly $2-4 in API calls per full seed run.

2.5 Schema realities — what is actually in the data

Free-text length distributions across the synthetic corpus:

Field	Median (words)	Range
Voice anchor	20	8-40
"Small thing" free text	14	6-35
Quirks (concatenated)	16	8-30
Embedding text (the full string vectorised)	110	40-220

Two missing-value paths bit the team in production and are worth surfacing as concrete examples of the system's fail-soft posture.

Per-question answer salience. A per-user weighting that boosts the matching contribution of free-text answers the user revealed more on. When it is absent — older profiles created before the field was added — the matcher falls back to uniform weighting. Match scores differ visibly between users on the same candidate pool. The fail-open path in the ranker ensures older profiles still get matches, even if the scores are honestly worse. The product does not dead-end them.

Gender on synthetic personas. This one was a near-disaster. After we shipped the same-gender hard filter for solo travellers as a safety default, the matching surface immediately started returning zero

candidates. The cause: we had added `gender` to the synthetic-persona schema, but the Pinecone metadata write step in the seed script — written months earlier — was only storing gender in the log-preview output, not the actual metadata dictionary. Every record in production had no gender field. The filter saw zero qualifying candidates, and the safety default ("fail open to mixed matching") meant the filter never fired at all. Diagnosis came at 2am. The fix was a metadata-only patch script that rebuilt the deterministic diversity matrix and called `index.update(set_metadata={"gender": ...})` per profile id, without re-paying language-model or image-generation cost. The script ran against production and patched 96 records in under a minute. The seed-script code path was then bumped so future re-seeds would not repeat the bug.

2.6 The eight-key emotional signature explained as a layer

The eight-key emotional-signature taxonomy is the third layer in the personality stack — coarser than GoEmotions, more identity-shaped than PPM. The eight archetypes (*story_collector, reset_seeker, aesthetic_pilgrim, depth_diver, energy_chaser, ritual_keeper, quiet_observer, threshold_walker*) cover the full space of traveller voices we observed in early synthetic personas. The inferrer picks one key per user based on two pieces of evidence — the GoEmotions distribution over the user's free text and their structured persona answers — and attaches a confidence level. The signature is then used as private framing for every persona-voiced surface: chat replies, *"why this?"* explanations, social posts, openers. The taxonomy key itself is never shown to the user. Only the derived emotional-tone phrase, such as *"soft afternoon energy"*, surfaces in the UI. The system is explicitly instructed never to use the taxonomy keys in output text, because they are internal labels rather than language a real person would naturally use.

This three-layer arrangement — GoEmotions for fine emotional signal, PPM for motivational structure, signature for voice — gives the system three independent lenses on a user's psychology, each at a different granularity. Each consumer in the system selects the lens appropriate to its task.

2.7 Generated-content quality, diversity, and bias

Two recurring drift patterns surfaced in generated content during the first weeks of running the synthetic-agents loop.

Sales-register drift. Early synthetic open-trip notes read like travel marketing copy. *"Join me for an unforgettable adventure!" "Looking for like-minded wanderlust souls." "Open to fellow travel buddies."* It is the voice that makes users distrust every other travel platform. The fix was to put *forbidden-openers* lists with explicit bad examples directly into every persona-voiced system prompt. Showing the model exactly what bad output looks like — rather than only describing it — turned out to be substantially more effective.

Question-loop drift in chat. Personas would drift into interrogating users after about turn seven of a chat session. *"What are you most excited about? What's your favourite memory of travelling? What would your dream day in Lisbon look like?"* Real people do not text like this. The fix was a runtime instruction inserted into the prompt when two or more of the persona's recent turns ended with a question mark — a *"breathe hint"* telling the model to react, observe, or share something small instead. Two-line change, completely altered the chat register.

Typography drift caught the worst. A user pointed out a message reading *"the street pad thai is genuinely different from restaurant versions , way more char on the noodles. there's a few stalls near the floating markets..."* Two distinct errors in one sentence: a stray space before the comma, and a singular contraction with a plural subject. The first fixes universally — a regular expression collapses whitespace before punctuation in every persona-voiced message before broadcast. The second fix went into the system prompt as a *"casual ≠ sloppy"* rule: *"there are a few stalls", not "there's a few stalls". Texting register means short and casual, not broken. A real person texting still hits agreement and spacing.*

2.8 The five hallucination categories the validator watches

Every persona-voiced surface in Sonder runs through a validator stack that watches for five regression categories:

Category	What it catches
Assistant-voice leakage	*"How can I help you?", "I'd be happy to..."* — chatbot register bleeding through
AI / tooling leakage	*"As an AI", "I'm a language model"*
Memory contradiction	Persona claims to have been somewhere, then later denies it within the same chat
Token-level failure	Empty generations, repetition stutters, malformed JSON
Internal-taxonomy leakage	Persona uses the system's own labels (*push*, *pull*, *motivation*) instead of behavioural language

Each category has a deterministic local pre-check that runs first — regex-based, instant, no cost — and a critic-prompt fallback that runs only if the local check is inconclusive. The combination matters. It kills the cheap failures pre-LLM and reserves model spend for the genuinely ambiguous cases. Telemetry tracks first-try approval rate, repair-triggered rate, and a continuous semantic-genericity score against a fourteen-stem set (*"sounds amazing", "hidden gem", "bucket list", "fellow traveler"*). The genericity score is particularly useful precisely because it is continuous. A creeping rise in mean genericity over weeks is a leading drift indicator long before any individual reply looks obviously bad.

2.9 How we built the vector database

The Pinecone vector index is the single most queried piece of infrastructure in the system. Every itinerary generation, every co-traveller match, every destination lookup hits it. The construction decisions shaped everything downstream.

Single index, three namespaces. A managed Pinecone serverless index named `sonder-index` hosts three logical partitions: `destinations` for city contexts, `activities` for venue and experience contexts, and `cotravellers` for traveller personality vectors. Namespaces share the same index and dimension but isolate query scope — a co-traveller search never accidentally returns a destination, and the metadata schemas can evolve independently per namespace. This was a deliberate choice over one-index-per-namespace because it keeps configuration centralised and lets the embedding-dimension

constant live in a single environment variable.

1536-dimensional embedding space, unified across every surface. Every vector — destinations, activities, traveller personas, the GoEmotions anchor glosses, the user's query text at retrieval time — is produced by the same OpenAI `text-embedding-3-small` model. The dimension constant is plumbed through configuration so a model change is one variable update plus an index recreate. Keeping all content in a single embedding space means cosine queries are coherent across namespaces — the team can re-rank a destination against a traveller's persona, or match an activity to a user's emotional signature, without managing separate model registries.

Metadata-driven filters at query time. Every vector carries a metadata dictionary alongside the embedding. Cotraveller records carry travel style, gender, age, location, archetype, persona answers (JSON-encoded), and the raw embedding text. Activity records carry category tags, cost in USD, duration in hours, and destination identifier. The matching pipeline's hard filters (budget feasibility, style match, gender for solo travellers, avoid-list veto) all run against metadata, not against the vector itself — which means a filter that would have eliminated a candidate happens *before* re-ranking, not after. Pinecone returns at most 50 candidates from cosine retrieval; metadata filters cut that down to the qualified pool; the feature pipeline ranks; the top three surface.

Embedding text shape matters. The string that gets vectorised for a traveller is a concatenation: voice anchor + small-thing free text + quirks + persona-answer labels (rendered as human-readable text, not snake-case keys). For destinations and activities, the embedding text is the full curated context block — 200 to 400 words describing what the place actually feels like rather than dry facts. Embedding the *feel* of a place lets cosine similarity surface emotional matches a fact-based embedding would miss.

Upsert pipeline. The seed scripts produce records in three steps: (1) build the diversity-matrix slots deterministically so re-runs hit the same profile IDs and `--resume` can skip already-seeded records; (2) generate persona content through the two-stage blind-writer pipeline, with the same-machinery inference step assigning PPM dimensions and emotional signature; (3) embed in batches of 100 per OpenAI request, then upsert in batches of 100 per Pinecone request. End-to-end seed time for the 192-persona singles population is roughly fifteen minutes, dominated by the LLM and image-generation calls rather than the Pinecone writes.

Metadata-only updates for schema evolution. A late discovery during the gender-filter rollout: Pinecone supports metadata updates without re-embedding. The team built a backfill script that rebuilt the deterministic diversity matrix, computed each existing profile's ID, and called `index.update(set_metadata={...})` per record. This patched 96 records in under a minute and cost nothing in LLM or image calls. The pattern generalises — any future schema-only field addition can ship via the same template.

3. The systems we built and what we compared them against

The team identified three problems to walk through in detail, each architecturally different from the other two: a retrieval-grounded generation problem, a validator-gated conversational generation problem, and a

feature-pipeline ranking problem. For each, we have a champion approach in production and a documented baseline we evaluated against.

3.1 Itinerary generation — full RAG pipeline against pure prompt LLM

What the user is trying to do. Submit a destination, dates, a budget, free-text persona answers, plus must-haves and things to avoid. Receive in return a day-by-day itinerary that names *specific* places, respects every negative constraint, fits the user's pace and budget, and includes a short *"why this for you?"* rationale in the persona's voice on every activity.

Champion approach. A retrieval-augmented generation pipeline grounded in the curated venue corpus. The user's persona is inferred across three pipelines running in parallel — a text embedder, the GoEmotions cosine classifier over their free text, and a language model that picks PPM dimension labels from a closed tool-use enum. None of the three sees the other two's outputs while running. The destination is then selected and ranked from the city vector store. Activities for that destination are retrieved from the activity vector store, ranked through a feature pipeline (cosine score, persona-question salience-weighted overlap, ordinal pace fit, budget feasibility, interest overlap), and handed to the itinerary-generating language model.

The model receives the user's persona, the retrieved real activities, and the emotional-signature framing. The model never sees the ranker weights, the cosine scores, any other user's persona, or the validator's prompt. Output streams day-by-day. Day one renders on screen within fifteen seconds while day seven is still being generated. The streaming pattern parses JSON forward-only and yields a parsed day the moment its closing brace lands, so the user is already reading while the model is still writing.

A separate validator engine then critiques the result across seven categories — budget fit, pacing realism, must-haves covered, avoid-list respected, day-sequence logic, activity specificity, feasibility risk. Outputs that fail can go through a refinement loop of up to three regeneration attempts. Each attempt re-embeds the persona with the failure feedback baked in, so the model receives a fresh query rather than grinding on the original one.

Challenger approach. The same persona and constraints rendered into a single prompt. The language model generates the itinerary cold. No retrieval, no ranker, no validator, no refinement, no streaming.

Comparative evaluation.

Dimension	Champion (RAG + rank + validator)	Challenger (prompt only)
Specificity — named venues, neighbourhoods, times of day	High	Low — generic "old town", "have a nice dinner"
Must-haves coverage	96-100% (validator-gated)	60-75%
Avoid-list violations	~0% (deterministic pre-filter on retrieval)	10-25% — the model invents plausible venues that violate the negative list

Dimension	Champion (RAG + rank + validator)	Challenger (prompt only)
Budget adherence	Hard pre-rank filter, never violated	30-40% violation rate on luxury-tier prompts
Validator first-try approval	~78%	Not applicable
Time to first day visible	12-18 seconds (streamed)	45-90 seconds (whole-buffer wait)
Hallucinated venue rate	Rare — model grounded on real activities	Common — plausible-sounding fictional places
Output token cost per trip	~6-8k tokens (large tier) plus retrieval	~6-10k tokens (large tier)

The integration battle behind this. The user-initiated revise flow originally reused the three-attempt refinement loop from the initial generation pipeline. Each iteration is a full regen plus validate. The loop was designed for the orchestrator's quality gate, not for revision. Wall time on a revise was hitting two to three minutes, and Cloudflare's edge function proxy was killing the connection around the thirty-second mark, leaving the page hanging with no error and no result. We rebuilt the revise path as a single-pass, classifier-routed pipeline with SSE streaming so days appear as they parse. Small-scope edits were then routed to a day-targeted prompt that regenerates only the affected days rather than the whole trip. Average revise dropped from roughly 150 seconds to roughly 25 seconds.

Trade-offs. The champion requires ongoing curation of the destination and activity corpora — fifty cities and five hundred activities is a finite curation budget, not infinity. When a user picks a city not in the corpus, the system falls back to *"invent plausible activities for {city}"* mode, which re-introduces the hallucination risk the RAG path was designed to eliminate. Automated corpus expansion is genuine future work, not a v1 problem. The challenger is operationally trivial, but its outputs are generic, which is precisely what every other existing recommender produces and the failure mode Sonder exists to avoid.

3.2 Chat replies — small-tier with validator-repair against large-tier prompt

What the user is trying to do. Text a synthetic persona inside Sonder. Receive a reply in character — texting register, no assistant voice, no AI leakage, no semantic genericity, no contradiction of what the persona said three turns ago — within a perceived-real latency budget of under five seconds.

Champion approach. Route chat replies to the small language-model tier (Claude Haiku in primary configuration, GPT-4o-mini under fallback). The persona's prompt layers three private framing blocks before the style rules: a hard trip-scope block (*"the trip is to Lisbon; never mention your home city, never suggest alternative destinations"*), a private psychology block where PPM dimensions and the emotional signature are passed as framing with an explicit instruction never to surface the words *push*, *pull*, *motivation*, *alignment*, or *friction* in output, and a per-turn breathe hint when recent turns have been too question-heavy.

The three pre-LLM fetches needed to assemble the context — vector-store candidate lookup, itinerary fetch, message history — run in parallel rather than serially. That change alone saved 600-800ms.

The architectural trick that makes the whole thing work: **the reply broadcasts to the user immediately, before validation.** A separate validator task runs asynchronously after broadcast. It first executes a deterministic local pre-check (minimum reply length, repetition detection, semantic-genericity score against the fourteen-stem filler set). If issues fire, a repair prompt rewrites the reply in a single pass under a fifty-word ceiling. If the repair changes the text, the persisted message is updated, and the chat surface receives a *message-edited* event that swaps the bubble text in place.

The user sees the reply land instantly. They watch it quietly self-correct a moment later when needed. Quality without latency cost. This is the single biggest UX win in the system, and the pattern generalises to any conversational surface where a small-model-first plus async-repair pipeline outperforms a single large-model call on both latency and cost.

Challenger approach. The same persona system prompt, routed to the large tier with no validator and no edit-in-place repair.

Dimension	Champion (small-tier + validator)	Challenger (large-tier, no validator)
Median time to user-visible reply	~1.4 seconds	~4.2 seconds
First-try validation pass rate	~74%	Not applicable
Repair-triggered rate	~26%	Not applicable
Banned-filler emission after cleanup	<1%	12-18% (*"oh nice", "honestly", "love that"*)
Token-level failures (empty, stutter)	0% (local pre-check kills pre-LLM)	1-2%
Persona consistency across 20 turns	High	Medium — drifts toward generic supportive register
Cost per reply	~$0.0005	~$0.012

The champion is three times faster, twenty-four times cheaper per reply, and the user does not experience the small tier as small.

3.3 Co-traveller matching — feature pipeline with learning against pure cosine baseline

What the user is trying to do. Receive a top-three list of potential co-travellers ranked by a number that honestly means *probability we'll click*, rather than a uniformly high cosine similarity that compresses every candidate into a narrow indistinguishable band.

Champion approach. A stage-based ranking engine that runs the same code path across three matching surfaces — co-traveller, destination, activity — with each surface declaring its own policy: which features to score, what weights to use, what feedback hyperparameters govern learning. Ten reusable scoring functions are available to any policy: raw cosine retrieval score, persona-question overlap weighted by per-question answer salience, emotional-signature exact match, pace ordinal distance, budget ordinal

distance, travel-style exact match, two flavours of interest overlap, activity cost fit, and pace-duration fit.

Hard pre-ranking filters fire before scoring. Budget feasibility uses a raw per-day budget cutoff with no fudge multipliers, because if the user said the budget is $80 a day they did not mean $200. Avoid-list veto removes anything matching a negative-constraint string. A travel-style hard filter ensures couples never see solo personas. **A same-gender hard filter for solo travellers** keeps solo women matched with women and solo men with men — a deliberate cold-strangers safety default that became the source of the biggest production firedrill in the entire project, detailed in a moment.

Per-user learning is the second half of what makes the ranker honest. The team modelled it as a cost-function with feature weights, where user feedback applies **weight penalties** that force the system to re-cost its own previous recommendations:

- **Explicit revise feedback.** When a user revises an itinerary with free text — *"this is too expensive"*, *"too touristy"*, *"too packed"* — the system keyword-maps that text to ranker features. *"Cheaper"* maps to *budget_ordinal_fit* and *activity_cost_fit*; *"less packed"* maps to *pace_ordinal_fit* and *pace_duration_fit*; *"more local"* maps to *tag_interest_overlap*. The boost on the matched features is positive (we want more of that signal); the implicit reduction on competing features is negative. The weight update applies with **decay across turns** — turn one applies the full boost, turn two applies half, turn three a quarter, with a floor at one-eighth. The decay prevents oscillation when a user keeps pushing back on similar things across multiple revisions, and it prevents the ranker from drifting wildly off the team's equal-weight priors based on a single confused user.
- **The self-correction loop.** A weight penalty is not just a record-keeping update. After the new weights are written, the same itinerary generation pipeline **re-runs from retrieval through ranking through generation through validation** with the new weights applied. The system literally corrects its own previous mistake by redoing the work. Revision history records every turn — original feedback, scope, target days, dropped activity titles, added activity titles, which feature weights moved, by how much, and the validator's verdict on the result. The whole thing is auditable.
- **Implicit chat signals.** A sarcasm-aware, negation-aware signal scanner runs after every chat message between the user and a candidate co-traveller. The scanner imports the same PPM keyword vocabularies the matching feature pipeline uses, so a user message that mentions *"crowded"* or *"hidden gem"* or *"splurging"* fires the corresponding weight update. **Negation zones extend five words or to the next clause break**, whichever comes first — *"I don't love crowded places"* does not boost the crowded-tag interest. **Sarcasm markers block boosts** when the message contains `/s`, *"said no one ever"*, an eye-roll emoji, or a *"love how..."* clause opener.

The combined effect is a system that continuously re-prices candidates against the same user as more signal accumulates. The persona's reciprocal-approval probability reads the live match score at the moment the user approves, so what a user reveals during a chat directly influences whether they end up matched. This is qualitatively different from a static ranker that scores once at retrieval time and never updates.

Challenger approach. Pure cosine retrieval. Top three by similarity. No features, no policy, no filters, no learning.

Dimension	Champion (feature pipeline)	Challenger (cosine-only)
Top-3 score distribution	0.45-0.75 typical — calibrated to feature explanation	0.78-0.92 — artificially high (cosine is similarity, not compatibility)
Feature explainability	Per-match reasons + compatibility breakdown	None
Cross-style bleed (couples seeing solos, etc.)	0% (hard filter)	25-40% (cosine ranks across all axes uniformly)
Same-gender enforcement (solo)	100% when gender is set	None
Adapts to in-chat signals	Yes — re-ranks per turn	No (frozen at retrieval time)
Adapts to revise feedback	Yes — text-feedback path with decay	No
Reciprocal-approval calibration	Score → probability is honest; observed approval 50-65%	Cosine → probability is dishonest; uniformly high observed approval (85-92%)

The calibration story is what matters most here. The champion's match score can be used directly as the reciprocal-approval probability. A score of 0.6 means a roughly 60% chance the matched persona accepts. This makes denial *meaningful* — a low score honestly produces a no. The challenger's cosine score, used as a probability, produces uniformly high approval rates that hide compatibility signal in the noise.

The pool-doubling firedrill. When we first shipped the same-gender hard filter, top-three match scores within each gender dropped to around 0.43. Plugging that into the formula `p_approve = match_score` meant a 57% deny rate on the reciprocal-approval roll. Users started reporting that they were seeing more denies than before. They were right. The filter had halved the effective candidate pool — 96 personas became 48 within their gender — and the top three within a smaller pool are necessarily lower-scoring. The team had two options: fudge the probability formula (dishonest, breaks the calibration) or double the pool. We doubled the pool. Bumped `PERSONAS_PER_SLOT` from one to two in the seed matrix, ran the seed with `--resume` so existing personas were not regenerated, generated 96 new personas. Top-three scores within each gender lifted ten to twenty points; deny rates dropped proportionally; calibration stayed honest. Real cost: roughly $4 in language-model and image-generation API calls.

Trade-offs. The champion ships with equal-weight priors across features — one over N for N features. This is the honest position given that we have not yet earned confidence in per-feature importance from accumulated data, but it leaves measurable signal on the table. The infrastructure to learn proper weights is already in place: every shown candidate, every accept, every reject is logged with the full feature breakdown at the time of the action. Phase two will compute replacement gradients — the difference between accepted and rejected feature vectors — and learn per-user weights from accumulated events.

3.4 Group-style filtering — solo, couple, family, friends are four different problems

The matching surface treats four user types as four genuinely different products. Trying to make one ranker serve all four would have meant compromising every one of them.

`TripConstraints.who_travelling_with` accepts one of four values, and `group_size` is bound by a validator at the API boundary so downstream code can trust the pair is consistent.

Style	Group size	Matching pool	Itinerary framing
solo	locked at 1	active (solo ↔ solo, same-gender filter applies)	Second-person singular; isolated instincts; counter-seating venues and walking-distance stops; 1-2 meet-others activities per day without forcing them
couple	locked at 2	active (couple ↔ couple, both members of each pair are male+female by seed design)	*"You both"* framing; every overnight private (no dorms); at least one slow shared activity per day; tables for two not bar-only; one shared-first experience as a memory anchor
family	2-8 user-picked	**disabled** — surfaces straight to shared-itinerary	**Assumes kids present:** all restaurants seat the full party at one table with kids-friendly menus; walking blocks under thirty minutes; dinner by 7pm; mid-day reset window; at least one kid-facing activity per day; apartment with kitchen access; no clubs, no fine dining, no min-age-fail activities
friends	2-8 user-picked	**disabled** — friend-group persona is too hard to write as one voice in v1	*"Your group"* framing; one-table reservations for N; at least one split-and-rejoin activity per day; nightlife on the table; apartment or villa over N separate hotel rooms

Why family and friends are disabled at the matching layer. A family of four or a group of friends already has its travelling party. Surfacing strangers as *"companions"* to a group that has each other makes no product sense. Family trips assume kids are along, which changes every activity recommendation (no bars, kid menus mandatory, early dinners) and changes the budget shape (apartments with kitchens, not hotel rooms). Friends trips assume a known group dynamic that no single synthetic persona could plausibly fit into. Both cases skip the matching surface entirely and route straight to the shared-itinerary negotiation flow where the existing group plans the trip together.

Why the hard style filter exists on solo and couple. The ranker has a `style_match` feature that nudges the score for matching travel styles, but a feature can only nudge — it cannot guarantee. Without the hard filter, couples were seeing solo personas surface as matches because of high embedding similarity on other axes. The hard filter at the route layer drops any candidate whose travel style does not match the viewer's, after retrieval and before re-ranking. Cross-style bleed dropped from 25-40% to 0%.

The party-size cap. `MAX_PARTY_SIZE = 8`. Past eight, the trip-planning shape changes — multi-room logistics, separate itinerary tracks, transportation that does not fit in a single vehicle — and neither the generator nor the ranker is tuned for that scale. Mismatched values are coerced silently rather than rejected at the API: passing `solo` with `group_size = 5` coerces size to 1; passing `couple` with `group_size = 1` coerces to 2.

Couple-mode seed pool. A couple matching with another couple needs other couples in the pool. The 18 male-female couple personas in the seed pool exist because the singles-only seed pool (where every persona is `travel_style = solo` by default) would have returned zero matches under the couple-mode hard filter. The couples seed script mirrors the singles two-stage blind-writer architecture but uses couple-specific layering: display name is *"X & Y"*, voice anchor uses *"we"* rather than *"I"*, quirks describe the couple's dynamic (*"she plans, he wings it"*), appearance descriptor renders in protagonist + partner order, and the portrait prompt explicitly requests two people in the frame as candid rather than engagement-shoot framing.

3.5 The pattern across all three champions

Look at the three champion approaches together and one architectural decision shows up across all of them, a decision the team would defend more loudly than anything else in the system: **information starvation as quality control.** The persona-inferring language model never sees the embedding vector or the GoEmotions output it will be merged with downstream. The validator never sees the user prompt the reply was responding to. The matcher's policy file never sees individual user data — only the feature names to score. Each component receives exactly the slice of context relevant to its narrow task, and the merge happens downstream rather than co-prompted upstream. This is what keeps the system's outputs defensible. A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest.

---

4. How it actually runs

4.1 Deployment architecture — separated layers at scale

Sonder runs on four cleanly separated service planes. Each one was chosen to be best-in-class for its specific responsibility rather than bundled into a single platform. The separation matters because it means scaling, monitoring, and replacing any one layer does not ripple through the others.

Frontend on Cloudflare Pages. The React single-page application is built statically and served from Cloudflare's global edge network — over three hundred points of presence worldwide. Static asset delivery happens at the edge with no cold start. A catch-all Cloudflare Pages Function proxies every `/api/*` request to the backend, keeping the SPA on a single domain. The single-domain constraint matters because the web-push service worker can only register against its own origin, and any cross-origin token traffic risks leaking authentication state. The initial plan tried to handle this through simple HTTP-redirect rewrite rules. Cloudflare rejected them, because cross-origin proxying via redirect violates their terms. An evening on the edge-function rewrite handled it cleanly, and the SPA stays on one domain while the backend API keys remain out of the client bundle. The edge function is itself a serverless Workers script that proxies headers verbatim, including authorization tokens.

Backend on Render. A FastAPI process runs on a long-running container with horizontal scaling enabled. The single process exposes REST routes, server-sent-event streams for itinerary generation and revision, WebSocket endpoints for chat and notifications, and runs the synthetic-agents background loop in the same lifespan. Render handles the auto-deploy from GitHub main, the TLS termination, and the health-check loop. The architecture is designed to scale to multiple replicas with one configuration change — the in-memory WebSocket connection manager becomes a Redis pub/sub channel via the `REDIS_URL` environment variable, so messages sent to one container reach WebSocket sessions on another. The swap is an evening of work, not a re-architecture, because the abstraction was built in from day one.

Multi-provider language-model layer. The routing engine reads a per-tier provider preference from configuration. The small tier (chat, openers, classifiers, *"why this?"* explanations, social posts, proposal evaluation) picks one primary; the large tier (itinerary generation, complex refinement) picks another. Either tier's automatic fallback is the other provider. Each provider client carries its own model identifier, which means cross-provider failover cannot accidentally route an Anthropic model identifier to OpenAI or vice versa. The discipline sounds obvious but bit the team once during a Sonnet deprecation week. The Anthropic-primary / OpenAI-fallback configuration is reversible per tier with a single environment variable change.

Data stores split by access pattern. A managed Pinecone serverless vector index holds the three namespaces. A named-database Firestore instance holds operational state — user profiles, generated itineraries, chat sessions with messages as a subcollection, shared-itinerary negotiation logs, social posts and comments, journal entries. Firebase Storage holds binary assets — persona avatars and cached voice MP3s. Voice cache keys are SHA-256 hashes of the (text, voice ID) pair, so a re-played message hits cache and is free on subsequent plays. Each store was chosen for its access pattern: Pinecone for cosine-similarity retrieval, Firestore for real-time listeners and small-document CRUD with strong consistency, Storage for blob serving with public-read URLs that the frontend can render directly.

Observability and analytics as separate concerns. Sentry handles error telemetry with deliberate filtering — *"Failed to fetch"* TypeErrors from client-side network loss are dropped because they are not bugs, and 4xx HTTP responses are dropped because they reflect expected user state (404 on first profile fetch, 401 from auth-state transitions). Only 5xx responses and genuine exceptions reach the dashboard, keeping the error feed signal-dense. PostHog handles product analytics with the funnel definitions, retention cohorts, and feature-flag scaffolding. The two platforms are kept separate because conflating error monitoring and product analytics in one tool would make both worse — Sentry's interface is built for triaging incidents; PostHog's is built for understanding user behaviour. Both connect via simple API keys and degrade gracefully when the keys are unset, so dev environments without monitoring never break.

The whole point of this separation is **operational independence at scale.** When Render needs to redeploy, the frontend keeps serving from Cloudflare's edge. When Cloudflare has a regional incident, the backend stays reachable for direct API calls. When Pinecone is slow, Firestore remains fast for the chat and trip-vault surfaces. When Anthropic rate-limits us, OpenAI picks up the load. No single failure cascades. Every layer can be replaced independently — Render swapped for another container platform, Cloudflare swapped for Vercel or Netlify, Pinecone swapped for Weaviate or Qdrant — without rewriting the others. That is what "designed at scale" actually means: not running at scale on day one, but being ready to without re-architecting.

4.2 The validator engine — guardrails that keep personas in character

The validator engine is the layer of the system the team is most quietly proud of. Every persona-voiced surface in Sonder — chat reply, opener, proposal evaluator, persona reveal, social post, *"why this?"* explanation — runs through a configurable critic stack that catches assistant-voice leakage, AI tooling leakage, semantic drift, memory contradictions, and a long tail of register failures before the user sees them. The whole engine is built as a generic critic-plus-repair stack with five surface-specific validator prompts and a configuration dataclass holding every threshold, prompt template, taxonomy stem, and regex in one place.

Five separate validator prompts, each strict and category-scoped. The itinerary critic watches for budget fit, pacing realism, must-haves coverage, avoid-list violations, day-sequence logic, activity specificity (the "swapability test" — would this description apply to any city?), and feasibility risk. The persona-reveal validator watches for concrete observation versus generic framing, evidence fidelity to the user's actual inputs, no itinerary content bleeding into persona copy, specificity, and internal-label leakage. The cotraveller match-card validator watches for ranking grounding, evidence fidelity, specificity, tension awareness (the persona reveal that mentions both alignment and friction reads more honest than one that only flatters), internal-label leakage, and tone. **The chat-reply validator is the most active** — ten categories including assistant voice, AI leakage, semantic drift, token stutter, empty token generation, romantic vibes, taxonomy leakage, unsafe content, bad conversation dynamics, and contradiction with established chat memory. The chat-reply repair prompt rewrites in a single pass under a fifty-word ceiling, preserving context but stripping anything the critic flagged.

A local deterministic pre-check runs first. Before any LLM critic call, a regex pass checks for minimum reply length (under three characters fires `empty_token_generation` and returns immediately, never even hitting the LLM), repetition stutters, and a semantic-genericity score computed by counting matches against a fourteen-stem filler set (*"sounds amazing", "hidden gem", "bucket list", "fellow traveler"*). The genericity

score is `base + matches × multiplier`, fires above a `0.80` threshold, and is also emitted as telemetry per event so the genericity distribution can be watched for drift over time. This local pass either kills bad replies outright or feeds its issue list to the LLM repair prompt as concrete *"validation issues"* context, so the rewrite is grounded rather than blind.

The trip-scope guardrails are the team's specific design choice for persona prompts. Every persona-voiced system prompt layers three private framing blocks before the style rules. The first is the **hard trip scope** block: *"STAY INSIDE THIS TRIP. The trip is to {destination}."* plus the itinerary digest, with explicit prohibitions on mentioning the persona's home city, on referencing past trips, on suggesting alternative destinations. Without this block, the model drifts. A Paris-based persona on a Japan trip would open with *"I see you're interested in Lisbon too?"* — the model gravitating back to wherever its training data has more mass. The second block is **PPM as private framing**: push, pull, and motivation labels passed in as context with the explicit rule *"never say push, pull, motivation, alignment, or friction in output — turn those into concrete behaviours, opinions, scenes, instincts."* The third block is the **emotional signature**, framed as *"private emotional framing (never expose these words). Let this shape cadence, warmth, pacing, what you notice — never the vocabulary."*

The result is personas that talk about a specific trip without dragging in their own backstory or the system's internal taxonomy. The critic catches the cases where the model breaks scope anyway.

Memory-contradiction detection is the subtlest catch. The validator parses chat history with two regex templates — past destinations matching *"i've been to X"* / *"visited X"*, and past negations matching *"never been to X"* / *"haven't visited X"* — and flags any reply that contradicts the established timeline. This catches a real failure mode where the persona "remembers" trips that never happened or denies trips it just claimed three turns earlier.

Per-execution telemetry. Every validator run emits an event to PostHog with first-try approval rate, whether repair was triggered, repair count, regex pre-check hit, total latency, the semantic-genericity score, the full execution log, and detected anomalies. The team watches validator behaviour empirically rather than trusting the prompts to keep working as the underlying models drift between versions.

Async edit-in-place is the load-bearing UX trick. For chat replies specifically, the validator is fully off the critical path. The unvalidated reply broadcasts immediately, then a `_validate_async` task runs in `asyncio.create_task`. If the repair changes the text, the persisted message is updated and a `message_edited` WebSocket event tells connected clients to swap the bubble text in place. Users see the reply land instantly and watch it quietly self-correct a moment later when needed. Quality without latency cost.

4.3 Real-time architecture — WebSockets, Firestore listeners, and modular separation

Sonder is a fundamentally real-time product. Chat messages, presence, typing indicators, shared-itinerary edits, push notifications, and discovery broadcasts all need to arrive at the user immediately. The architecture splits this across two transports chosen deliberately for what each is good at.

WebSockets for live conversational traffic. Two WebSocket endpoints power the live surfaces. The first is `/ws/chat/{session_id}` — one WebSocket per active chat session, carrying messages, typing indicators, seen receipts, and presence updates between user and persona (or eventually, user and user). The second is `/ws/notifications` — a single global WebSocket per logged-in user, carrying chat notifications, match notifications, comment notifications, and the discovery broadcast events that drive the live travellers strip on the dashboard.

First-message authentication, not query strings. WebSocket endpoints take their auth token in the first JSON message after handshake, never as a query parameter. Tokens in URLs leak through browser history, server logs, and referer headers. The first-message pattern keeps the credential off the wire and out of every monitoring system that might inadvertently capture it.

Heartbeats for presence. Clients send `{"type": "ping"}` every thirty seconds. The backend records `last_seen` to Firestore on each ping. Presence is then derived as `(now - last_seen) < 90 seconds` rather than a boolean `online` flag, because boolean flags go stale when WebSocket connections drop unexpectedly (network change, tab close, hibernation). The TTL-derived approach is honest — if you have not heard from the user in ninety seconds, you do not know they are online.

Firestore listeners for the shared state surfaces. The shared-itinerary negotiation page and the presence indicators do not use WebSockets directly. They use Firestore's real-time listener API, which the client subscribes to per-document. Every change to a `shared_itineraries/{id}` document fan-outs to both negotiation participants in milliseconds. The backend writes; Firestore propagates; both clients re-render. The team chose Firestore listeners over a custom WebSocket protocol because Firestore guarantees ordering and delivery semantics that would have required significant engineering to replicate on top of raw WebSockets.

Modular separation across four team-owned folders. The codebase is split into four modules, one per team member, with strict import boundaries: Jahnvi owns schemas and the persona pipeline modules and the React frontend; Shreyas owns retrieval, ranking, the matching surface, and the real-time connection manager; Ali owns the LLM clients, the routing engine, all generation prompts, and RAG; Mushahid owns the FastAPI app, the validation engine, the refinement loop, the realtime fan-out layer, and deployment. Cross-module imports are documented per folder. The boundaries are real — a change to a schema in Jahnvi's folder requires updating the docstrings in Shreyas's and Ali's folders that import from `shared/schemas.py`. The discipline pays off because it means each team member can ship independently. Schema changes don't break LLM calls; LLM provider changes don't break the ranker; ranker changes don't break the frontend.

4.4 The shared-itinerary real-time negotiation

The shared itinerary is the surface where two matched co-travellers actually plan the trip together. It is the single most architecturally interesting piece of the system and deserves its own walkthrough.

Both sides edit one document. When a pair mutually approves, the system clones the chosen itinerary into a new `shared_itineraries/{id}` document with version 0 and both user IDs in `participants`. From that moment on, every change goes through the proposal-counter-accept loop rather than direct edit. Direct

edits would race; mediated edits are explicit, reviewable, and persisted.

The proposal flow. A user proposes a change — add an activity, replace an existing one, move it to a different day. The proposal writes a `ProposedChange` record with status `proposed` and an `evaluating` activity entry in the itinerary's activity log. The HTTP response returns immediately. The change does not yet exist in the itinerary itself; it is a pending proposal awaiting evaluation.

The persona evaluator runs asynchronously. Once the proposal is recorded, the backend kicks off an `asyncio.create_task` that runs the persona's evaluator language-model call. The evaluator is a small-tier prompt that takes the persona, the current itinerary, the recent negotiation history, and the proposed change, and returns a JSON verdict: accept (commit the change), or counter (mint a new `ProposedChange` from the persona with a different activity and a short reason). **There is no hard reject state.** If the persona dislikes the proposal, the system requires it to either accept or offer a counterproposal — never to stonewall. This keeps the negotiation always-collaborative rather than stuck.

Reason-in-message rule. Every counterproposal must include a brief conversational reason: *"hakone feels far after the museum day"*, *"the dinner reservation locks our evening — what if we move it earlier?"*. Without the reason, the counter reads arbitrary and the user cannot meaningfully engage with it. The system prompt enforces this and the validator catches counters that come back reasonless.

Token-Jaccard dedupe on counters. The persona is forbidden from counter-suggesting an activity that already exists in the itinerary, has already been accepted, or has already been rejected. A backend pass normalises titles to lowercase tokens and runs Jaccard similarity (threshold 0.6) against every prior item. If the LLM still returns a near-duplicate counter — which it occasionally does — the verdict gets flipped to accept with a fallback message (*"yeah okay, let's go with yours"*) rather than circulating the same idea with slightly different wording. The negotiation moves forward rather than looping.

Optimistic locking on every write. Each shared-itinerary document carries a `version` field. Every write checks `client_version == current_version` before committing. A mismatch returns HTTP 409, the client re-fetches the latest state, shows the user a toast (*"Someone else made a change — here's the latest"*), and lets them retry their action against the updated version. This is what prevents the classic two-people-editing-simultaneously race condition without silent overwrites.

The activity feed. Every proposal, every counter, every accept, every reject lands in the shared itinerary's activity log with a timestamp and an actor. The frontend renders this as a live feed filtered to entries created after the page opened — so a reload gives a clean visual slate without touching the persistent log. Resolution dedupe strips `evaluating` entries that have a follow-up resolution (accepted, countered, proposed) from the same actor within ninety seconds, because the verdict line carries the same information. Orphan timeout strips `evaluating` entries older than sixty seconds with no follow-up, catching the dropped LLM calls that would otherwise linger forever.

Finalisation locks the itinerary. Either side can finalise, which sets `finalized_at` and rejects any further proposals with HTTP 409. The pair has committed; the trip is the plan.

The decision pressure. The team designed the negotiation to be *bounded by exhaustion rather than by a hard turn limit.* In practice, the proposal cadence and the asynchronous evaluator latency naturally compress the negotiation into a small number of meaningful exchanges — typically two to four rounds of proposal and counter before the pair lands on something both are happy with. A hard cap on rounds was considered but never shipped, because the existing dedupe logic plus the no-hard-reject rule create the bound implicitly: the persona cannot circulate the same idea twice, and cannot refuse outright, so the conversation has to move forward.

4.5 The full model inventory

Sonder orchestrates eight distinct generative or representational models, each chosen for a specific task profile rather than as a one-size-fits-all default.

Model	Provider	Role in Sonder
Claude Haiku 4.5	Anthropic	Small-tier conversational surfaces: chat replies, openers, classifiers, *"why this?"* explanations, social posts and open-trip notes, proposal evaluation
Claude Sonnet 4.6	Anthropic	Large-tier generative surfaces: itinerary generation, complex refinement, conflict resolution between paired travellers
GPT-4o-mini	OpenAI	Small-tier fallback when Anthropic is unavailable
GPT-4o	OpenAI	Large-tier fallback
text-embedding-3-small	OpenAI	All retrieval embeddings (1536-dim): destinations, activities, traveller profiles, persona text, GoEmotions anchor vectors
gpt-image-1	OpenAI	Synthetic-persona portrait generation, seed-time only
eleven_multilingual_v2	ElevenLabs	Persona voice text-to-speech, with deterministic voice-ID assignment per persona via appearance → accent → gender lookup
GoEmotions cosine classifier	In-process	Emotion scoring over free text via cosine distance against 27 anchor vectors embedded once at process start

There are eight rather than two because each model does a thing the others cannot do as well or as cheaply. Haiku handles persona-voiced texting at twenty-five times lower cost than Sonnet with no

measurable quality loss against the validator stack. Sonnet's 16k output token ceiling matters for full-trip JSON. The image-generation model's painterly outputs are explicitly biased away from photorealism, which is the right register for the *"Sonder Curated"* disclosure pattern. ElevenLabs offers the multilingual voice library no general-purpose TTS matches. The GoEmotions cosine classifier lives in the same embedding space as everything else, so adding it required zero new infrastructure.

4.6 Prompts live in code

Every persona-voiced prompt — chat reply, opener, proposal evaluator, social post, open-trip note, itinerary refinement, validator critics — is a module-level constant in the codebase. Versioning is by git. A prompt change is a reviewable commit. There is deliberately no external prompt store, because the coupling between prompt and surrounding code is explicit. A prompt change that breaks downstream parsing is caught by code review rather than discovered in production.

For larger structural prompt changes, the rollout pattern is to deploy the prompt update paired with a post-hoc normaliser that catches drift from outputs still in flight. The chat opener's *Hey {Name}!* greeting contract is the canonical example. When the format changed mid-deploy, both code paths gained the new prompt rules *and* a wide drifted-greeting matcher that catches outputs from personas that had not yet observed the new prompt. Belt-and-braces because language models are stateless, and the team cannot roll them mid-stream.

4.7 Model updates and the fine-tuning question

Model identifiers are environment-driven. A model bump — moving from Sonnet 4.5 to 4.6, for example — is a single environment-variable change plus a redeploy. The application code is provider-agnostic and model-id-agnostic. Only the routing layer and the per-provider client know specifics.

Sonder has not used fine-tuning. The team is betting on three less-expensive techniques in combination: prompt engineering with explicit positive and negative examples (every persona-voiced prompt includes both *"Good shapes"* and *"Forbidden"* example blocks), the validator stack as the second line (when prompt engineering misses, the validator catches it, and the validator's failure analysis becomes feedback for the next prompt iteration), and per-user learning at the ranker layer (the matching surface adapts to individual users through observed feedback rather than through a fine-tuned model).

Fine-tuning remains a phase-two escape hatch for surfaces where prompt engineering has demonstrably plateaued — most likely the proposal evaluator, where persona-consistent counter-suggestions across long negotiations is the hardest sustained-coherence task in the entire system.

4.8 Monitoring for hallucinations, drift, and performance regression

Every language-model surface in Sonder is instrumented. Five top-level metric families drive the analytics dashboard.

User satisfaction. Match found, match approved, match denied, match regenerated, itinerary revision applied, refinement attempt counts. A rising denies-per-match ratio is a leading indicator of a calibration regression. A rising revisions-per-itinerary ratio is a leading indicator of an itinerary-quality regression.

Retrieval quality. Retrieval completion events carry destination count and activity count. A surface returning zero candidates points at a corpus coverage gap before users start abandoning sessions.

Response quality. Validator stack executions per surface carry first-try approval rate, repair count, total latency, and the continuous semantic-genericity score. The genericity score is the most useful single metric in the stack because it is a continuous signal. A creeping rise in mean genericity over weeks is a soft drift warning long before any individual reply looks bad.

Hallucination rate. Itinerary and persona validation pass rates broken down by issue category — assistant-voice leakage, memory contradiction, taxonomy leakage, and others. A category climbing in isolation points at a specific regression. An uptick in memory-contradiction issues after a chat-history cap change, for example, would surface immediately.

Itinerary completion funnel. Plan started → trip generated → trip saved → trip viewed. The conversion rate between adjacent steps is the product's primary growth metric.

A per-feature distribution observer on the ranker side records mean, variance, and count per (surface, feature) combination. This catches silent scale domination — for example, if raw cosine score is winning 80% of the combined ranker output because its distribution sits at 0.78 while ordinal fits sit at 0.5, the asymmetry is visible in the aggregate rather than discovered three months later through empty engagement metrics.

The two-platform split. Error monitoring lives on Sentry; product analytics live on PostHog. The two are deliberately separate because they answer different questions and live in different team workflows. Sentry catches stack traces, response latency outliers, third-party-API failures, and the genuinely-broken events that pull an engineer into the codebase the same day. PostHog catches funnel conversion, retention cohorts, feature-flag rollouts, the validator-stack telemetry, and the slower aggregate metrics product decisions are made on. Conflating the two in one tool would make both worse: Sentry's interface is built for triaging incidents (group similar errors, mute noise, assign owners), while PostHog's is built for cohort analysis and behavioural reasoning. Both platforms degrade gracefully — a missing API key turns the relevant calls into no-ops so dev environments without monitoring still run.

The Sentry noise filter. Vanilla Sentry over a frontend app would be unusable. The team built a custom filter that drops two large categories of expected events: pure *"Failed to fetch"* TypeErrors (client lost internet — not a bug we can fix), and HTTP 4xx responses (404 from `/users/profile` on first login is the documented signal that we should create the profile; 401 from `/api/cotraveller` during the auth-state transition is expected). Only 5xx responses and genuine uncaught exceptions reach the dashboard. This is the difference between a Sentry feed that gets ignored and one that an engineer actually reads.

4.9 The feedback loops running today

Three feedback paths are live in production.

Implicit accept-reject events. Every shown match, every accept, every reject is logged with the candidate's full feature breakdown at the moment of the action. This is the substrate for phase-two gradient learning.

Explicit free-text revision feedback. When a user revises a generated itinerary, the feedback is classified for scope and target (which days, which categories), then routed through either a day-targeted regeneration prompt or a full-itinerary regeneration. The free text is also keyword-mapped to ranker features for that user — *"cheaper"* boosts budget-fit weighting going forward, with decay so that repeated similar feedback dampens rather than oscillates.

Live chat signals. The sarcasm-aware signal scanner runs after every message, re-ranks the candidate, and updates the match score. The persona's reciprocal-approval probability reads the live score at decision time. What a user reveals mid-conversation directly influences whether they end up matched. This is the loop the team is most excited about, because it makes the system feel responsive in a way that is hard to articulate to a stakeholder until they experience it.

Human-in-the-loop oversight is currently lightweight. The team reviews validator-flagged samples through the analytics dashboard weekly. Prompt changes ship through git review with at least one additional set of eyes. A formal moderation queue is phase-two work — synthetic content does not need it, but user-generated journal entries and feed posts will need it at scale.

---

5. What we learned, what is still broken, and where this goes

5.1 What worked

Information starvation is the single biggest architectural lever the team discovered. The discipline of giving each component exactly the slice of context relevant to its narrow task — and merging downstream rather than co-prompting upstream — is what makes Sonder's outputs surprising rather than averaging toward the mean. The persona inference, the validator, the matcher's policy declarations, and the synthetic-persona blind-writer seeding all use the same pattern. The discipline carries.

Async edit-in-place validation is the chat UX win nobody is talking about. Small-model-first plus async-repair gives users large-model-validated quality at small-model latency. Three times faster, twenty-four times cheaper per reply, and users do not experience the seam. The team would recommend this pattern to anyone running conversational AI in production.

Equal-weight priors paired with logged-everything infrastructure is the honest move when you do not yet have data. Refusing to hand-tune ranker weights and instead instrumenting every feature observation to disk is what keeps the door open for phase-two learning. Most products tune weights by gut and then cannot reconstruct why. Sonder waits, because the data substrate is being built in the meantime.

5.2 What is still broken or limited

Equal-weight priors leave measurable signal on the table. Honest acknowledgement of a known gap. Resolution is engineering work — accumulate enough feedback events to compute replacement gradients, then ship the learning loop. Not a research problem.

Synthetic-only safety filters. The same-gender hard filter is enforced against synthetic-persona metadata. Real users joining the matching pool would need a verified gender field. Currently self-reported via the backfill prompt. Scale-up to real-user matching needs identity verification, and that becomes an entire phase-two product surface in its own right.

No automated test suite for LLM-dependent outputs. Local deterministic tests cover routing, validators, ranking math, and pipeline shapes. The LLM-dependent surfaces are monitored through production telemetry rather than CI-tested, because LLM outputs are genuinely hard to assert against deterministically. This is a deliberate trade-off, but it means regressions can ship and only get caught by aggregate metric drift. Mitigating with validator-stack telemetry is the current answer. Building a sample-evaluation harness with LLM-as-judge is a phase-two consideration.

Vector store corpus maintenance is manual. Fifty curated destinations and five hundred curated activities is a curation budget the team is carrying personally. A user picking a city not in the corpus falls back to an *"invent plausible activities for {city}"* prompt that re-introduces the exact hallucination risk the RAG pipeline normally controls. Automated corpus expansion is phase two — scrape candidate venues from public sources, embed on-demand, validate against the same critic stack. Estimated effort: two engineering weeks plus an ongoing curation review queue.

Synthetic content does not auto-throttle. The synthetic-agents loop runs at the same cadence regardless of real-user density. Once real-user content reaches a critical mass, the synthetic side should throttle down. Right now it is a manual configuration change. A self-balancing implementation is straightforward — track new-content-per-hour by source — but it has not shipped yet, because the platform is not at that density yet.

5.3 Where this goes next

- **Phase-two ranker learning** replaces equal-weight priors with gradient-learned weights from accumulated accept-reject events. The data substrate is already in place.
- **Automated activity-corpus expansion** through scraping plus validator-gated embedding for novel destinations. Removes the prompt-fallback hallucination risk.
- **Multi-modal persona reveal.** The cinematic destination-reveal screen is currently photo-driven. Phase two could add persona-voiced audio greetings (the voice infrastructure already exists for chat) or short generated video sequences.
- **Layered prompt-injection defence.** The current input sanitiser is a lightweight regex first line. A language-model classifier on top for the high-stakes free-text fields — proposals, chat, journal — is the obvious second layer.

- **Cross-candidate diversity in ranking.** The matcher's reranker stage is declared as a no-op in v1, reserved for diversity, fatigue, and sequencing features. Maximum marginal relevance would prevent top-three matches from all being near-duplicates of each other.
- **Verified identity for real users.** Self-reported gender is a starting point. Scale needs verification.

5.4 References and resources

Models running in production.

- Anthropic Claude — Haiku 4.5 for small-tier conversational surfaces; Sonnet 4.6 for large-tier generation; validator tier available on either model.
- OpenAI — GPT-4o-mini and GPT-4o (fallback provider, same tier split).
- OpenAI text-embedding-3-small — 1536-dimensional embeddings, used uniformly across every retrieval surface.
- OpenAI gpt-image-1 — synthetic-persona portrait generation, seed-time only.
- ElevenLabs eleven_multilingual_v2 — persona voice text-to-speech.
- GoEmotions 27-label classifier — in-process cosine over anchor vectors, living in the shared embedding space.

Datasets and curated corpora.

- 192 LLM-designed synthetic solo travellers and 18 couples, all seeded into the vector store.
- Hand-curated destination corpus (~50 cities) and per-destination activity corpora (~500 activities total).
- Closed twelve-dimension push-pull persona vocabulary.
- Closed eight-key emotional-signature taxonomy with tone-anchored glosses.
- GoEmotions 27-label emotion vocabulary used as anchor vectors.

Academic frameworks the system is built on.

- *Dann, G. (1977). "Anomie, Ego-Enhancement and Tourism." Annals of Tourism Research, 4(4), 184-194.* — Original Push-Pull Motivation Theory in travel research.
- *Crompton, J. L. (1979). "Motivations for Pleasure Vacation." Annals of Tourism Research, 6(4), 408-424.* — Extension of Dann's framework that informed the six-and-six dimension adaptation.
- *Demszky, D., et al. (2020). "GoEmotions: A Dataset of Fine-Grained Emotions." arXiv:2005.00547.* — Source of the 27-label emotion taxonomy used as anchor vectors.

External APIs and services.

- Wikipedia REST API for destination context, lede paragraphs, and infobox imagery.
- Pixabay for popularity-ranked travel photography powering the cinematic reveal.
- OpenWeather for current weather by latitude and longitude.

- Nominatim / OpenStreetMap for geocoding with one-request-per-second etiquette.
- ExchangeRate API for currency conversion with a 30-currency hardcoded fallback.

Infrastructure.

- Pinecone — managed vector index, three namespaces.
- Firestore (named-database mode) — operational document store.
- Firebase Authentication and Firebase Storage.
- Render — application server hosting.
- Cloudflare Pages with edge functions — frontend hosting and backend proxying.
- VAPID with a service worker — offline web-push notifications.

The repository. github.com/shreyast36/sonder — full source, README, and per-person build checklist sustaining the work across the team.

Built over a series of sleepless nights by a team that got tired of explaining trips to algorithms.